

No	Items
1	Introduction to Algorithms & Complexity Analysis
1.1	How do you understand the concepts of Time Complexity and Space Complexity of an algorithm? Why do we need to care about these two factors?
1.2	How do you understand Big O notation? What is it used to represent in algorithm analysis? Please provide an example of a function with a complexity of $O(n)$
1.3	Based on the pseudocode below, analyze and determine its Time Complexity using Big O notation. function find_sum(array): total = 0 for number in array: total += number return total
1.4	Compare the performance difference (based on Big O) between an algorithm with $O(n)$ complexity and one with $O(n^2)$ complexity as the input size (n) increases. Which algorithm do you think is more efficient?
1.5	Suppose you need to build a "find mutual friends" feature for a social media application. To do this, you have the friend list of person A and person B. Consider the following two approaches: + Approach 1 (Nested Loops): Iterate through each friend of A. For each friend, iterate through the entire friend list of B to check for a match. + Approach 2 (Using Extra Memory): Take A's friend list and put it into a temporary data structure that allows for very fast lookups (e.g., a Hash Set). Then, iterate through B's friend list and use that structure to check if each friend exists. In your opinion, which approach would consume more memory (Space Complexity)? Which one would return the result faster (Time Complexity), especially when each person has thousands of friends? To provide a smooth user experience, which approach do you think engineers would prefer? Why?
1.6	Think about a task you do daily (e.g., finding a book on a shelf, sorting your clothes) and try to describe the steps you take in the form of an algorithm. Then, estimate its "complexity."
2	Basic Data Structures
2.1	How do you understand the fundamental difference between an Array and a Linked List in terms of how they store data in memory and handle insertion/deletion operations?
2.2	How do you understand the principles of LIFO (Last-In, First-Out) for a Stack and FIFO (First-In, First-Out) for a Queue?
2.3	Which data structure (Array or Linked List) would you choose to store a list of songs in a playlist where users frequently add, remove, and reorder tracks? Explain your reasoning.
2.4	What mechanism does a web browser use to manage the pages you've visited when you click the "Back" button? Explain why a Stack is a suitable data structure for this functionality.
2.5	Simulate the operations of a Queue with the following sequence of requests: enqueue(A), enqueue(B), dequeue(), enqueue(C), dequeue(). Which element will be at the front of the queue after these operations?
2.6	Describe a real-life example that operates similarly to a Queue (e.g., lining up at a supermarket). What problem does applying the Queue principle solve in that situation?
2.7	Imagine you are building an "Undo" feature for a text editor. What data structure would you use to store the user's actions? Why?

3	Searching and Sorting Algorithms
3.1	How do you understand the difference between Linear Search and Binary Search? What is the prerequisite for applying Binary Search?
3.2	How do you understand the basic working idea of the Bubble Sort algorithm? Why is it generally not considered efficient for large datasets?
3.3	Compare the pros and cons of Merge Sort and Quick Sort. In which scenarios would you prefer Merge Sort over Quick Sort, and vice versa?
3.4	You have an unsorted list of 1000 students. If you need to find the information of a specific student as quickly as possible (and you will perform this search many times), what would you do with the list first? Which search algorithm would you use afterward? Explain.
3.5	The search function in your phone's contact list works very quickly, even with hundreds of contacts. Do you think the contacts are sorted, and could it be using a search algorithm similar to one we've learned?
3.6	When you sort emails in your inbox by date (newest on top), the email client is performing a type of sorting algorithm. If a new email arrives, where is it inserted? This process is similar to which sorting algorithm (e.g., Insertion Sort)?
4	Recursion (options)
4.1	How do you understand Recursion? What are the two essential components of a recursive function?
4.2	How do you understand the role of the Base Case in a recursive function? What would happen if a recursive function lacked a base case or never reached it?
4.3	The Tower of Hanoi puzzle is a classic example of applying recursion. Try to explain the main idea of moving n disks from peg A to peg C using recursive thinking.
4.4	Imagine you need to find a file in a folder that contains many nested subfolders. Describe how you could apply recursive thinking to solve this problem. What is the "base case" here?

Answer

[illegible]
